

高级数据结构第三次作业

学号：202533354 姓名：ZHOU MINGYUAN

1. 线性探测哈希表

哈希表大小为 10， $h(x)=x \bmod 10$ ，插入顺序为 12, 22, 32, 25, 35。

key	原始哈希地址
12	$12 \bmod 10 = 2$
22	$22 \bmod 10 = 2$
32	$32 \bmod 10 = 2$
25	$25 \bmod 10 = 5$
35	$35 \bmod 10 = 5$

探测过程：12 直接插入 2；22 探测 2 被占用，插入 3；32 探测 2、3 被占用，插入 4；25 直接插入 5；35 探测 5 被占用，插入 6。

0	1	2	3	4	5	6	7	8	9
空	空	12	22	32	25	35	空	空	空

2. 平方探测哈希表

哈希表大小为 11， $h(x)=x \bmod 11$ ，探测位置为 $(h(x)+i^2) \bmod 11$ 。

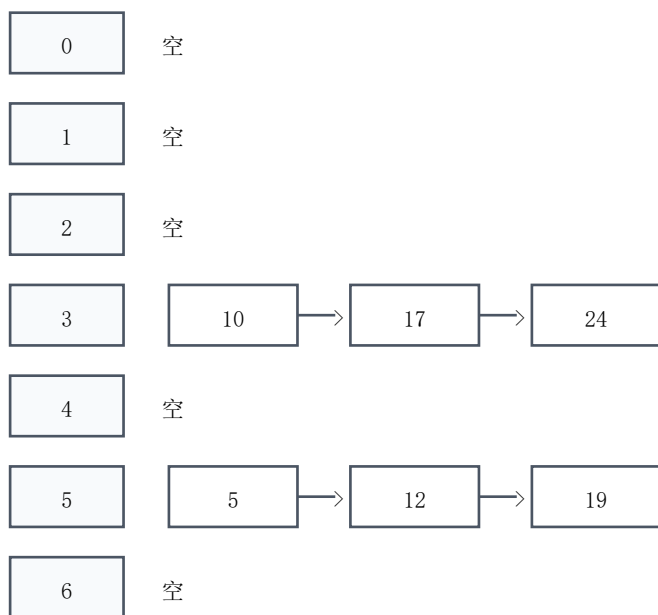
key	原始地址	平方探测过程	最终位置
22	0	0	0
33	0	0 被占用 $\rightarrow$ 1	1
44	0	0 被占用 $\rightarrow$ 1 被占用 $\rightarrow$ 4	4
15	4	4 被占用 $\rightarrow$ 5	5

0	1	2	3	4	5	6	7	8	9	10
22	33	空	空	44	15	空	空	空	空	空

平方探测的优点：相比线性探测，它不再连续检查相邻位置，能够减轻一次聚集现象，使冲突元素更分散。

3. 链地址法哈希表

哈希表大小为 7， $h(x)=x \bmod 7$ 。各 key 的 bucket 为：10 $\rightarrow$ 3, 17 $\rightarrow$ 3, 24 $\rightarrow$ 3, 5 $\rightarrow$ 5, 12 $\rightarrow$ 5, 19 $\rightarrow$ 5。



链地址法把冲突元素存放在同一 bucket

后面的链表中；开放式寻址法仍把所有元素放在表数组内，冲突时按照探测序列寻找其他空 slot。

4. 动态哈希

(a) 静态哈希 bucket

数固定。数据增长后装填因子升高，冲突变多，查找和插入成本上升；若一次性重建表，代价也较大。

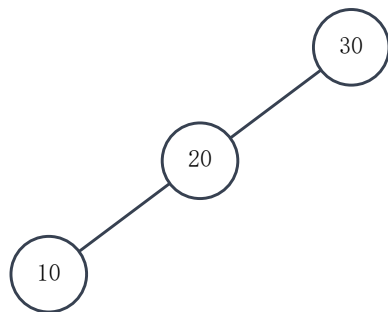
(b) 可扩展哈希使用目录和若干 bucket。目录由全局深度控制，bucket 有局部深度；当某个 bucket 溢出时只分裂该 bucket，必要时目录加倍。

(c) 线性哈希按分裂指针逐个 bucket 渐进扩容，通常不需要目录整体加倍；可扩展哈希按发生溢出的 bucket 分裂，并在局部深度超过全局深度时把目录加倍。

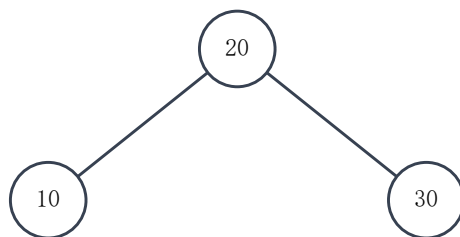
5. AVL 树插入与旋转

依次插入 30、20、10 后，30 的左子树高度比右子树高 2，且新节点插入在左孩子 20 的左侧，因此属于 LL 型失衡。

插入 10 后的失衡结构：

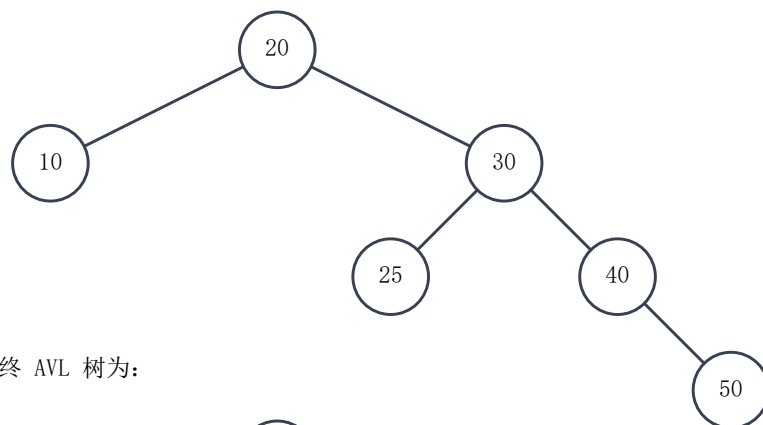


对节点 30 做一次右旋，旋转后树结构为：

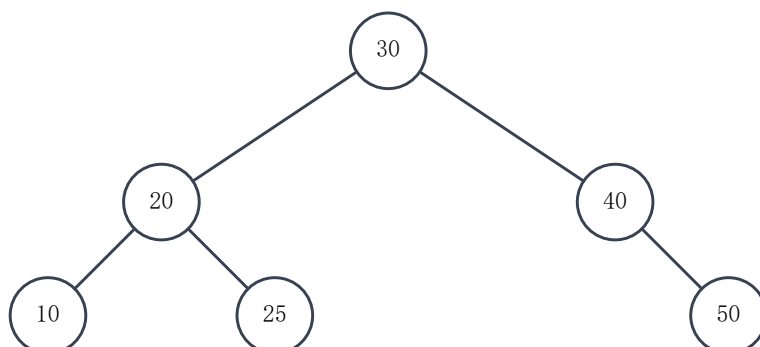


继续插入 25、40 后，各节点平衡因子仍在 -1 到 1 之间，不失衡。插入 50 后，节点 20 的右子树高度过高，且新节点位于右孩子 30 的右侧，属于 RR 型失衡。

插入 50 后调整前：



对节点 20 做一次左旋，最终 AVL 树为：



AVL 树相比普通二叉搜索树的优点：通过维护平衡因子限制树高，使查找、插入和删除的时间复杂度保持在 $O(\log n)$ ，避免普通二叉搜索树退化成链表。

6. AVL 树平衡因子

按 $BF(T) = \text{左子树高度} - \text{右子树高度}$ 计算。

- (a) $BF(T) = 3 - 2 = 1$ 。
- (b) 该节点平衡，因为 $|BF(T)| = 1$ ，不超过 AVL 树允许的范围。
- (c) 右子树高度变为 3 后， $BF(T) = 3 - 3 = 0$ 。
- (d) 若左子树高度继续增加为 5，则 $BF(T) = 5 - 3 = 2$ ，节点 T 失衡，需要旋转。具体是 LL 还是 LR，要看新节点插入在 T 的左孩子的左子树还是右子树中。

7. 红黑树插入调整

- (a) 新节点 u 和父节点 pu 都为红色，违反了红黑树中“红色节点不能有红色孩子”的性质，也就是不能出现连续红节点。
- (b) 叔节点为红色，因此属于 XYr 不平衡。
- (c) 调整方法：把父节点 pu 和叔节点染成黑色，把祖父节点染成红色；若祖父节点是根，则再把根染回黑色。之后若祖父节点与其父节点继续形成红红冲突，则向上继续调整。
- (d) 这种情况下通常不需要旋转，因为父节点和叔节点同为红色时，冲突可通过重新染色消除，同时保持各路径黑高不变；旋转主要用于叔节点为黑色的 XYb 情况。

8. AVL 树与红黑树判断题

小题	判断	理由
(a)	正确	AVL 树调整完成后任意节点都满足 $ BF \leq 1$ ，所以 BF 只能是 -1、0、1。
(b)	错误	红黑树新插入节点通常先染成红色，以避免立即改变根到叶子的黑高。
(c)	正确	红黑树的平衡条件比 AVL 树宽松，更新时通常需要的旋转次数更少。
(d)	正确	二者使用的单旋、双旋本质上都是二叉搜索树旋转；区别在触发条件和红黑树还要重染色。
(e)	错误	红黑树只保证近似平衡，不要求每一层都填满，也不一定是完全平衡二叉树。